
Batch Inference on Deep Convolutional Neural Networks with Fully Homomorphic Encryption Using Channel-By-Channel Convolutions

Jung Hee Cheon^{1,2} Minsik Kang¹ Taeseong Kim¹ Junyoung Jung¹ Yongdong Yeo¹

Abstract

Secure Machine Learning as a Service (MLaaS) is a viable solution where clients seek secure ML computation delegation while protecting sensitive data. We propose an efficient method to securely evaluate deep standard convolutional neural networks based on residue number system variant of Cheon-Kim-Kim-Song (RNS-CKKS) scheme in the manner of batch inference. In particular, we introduce a packing method called *Channel-By-Channel Packing* that maximizes the slot compactness and Single-Instruction-Multiple-Data (SIMD) capabilities in ciphertexts. We also propose a new method for homomorphic convolution evaluation called *Channel-By-Channel Convolution*, which minimizes the additional heavy operations during convolution layers.

Simulation results show that our work has improvements in amortized runtime for inference, with a factor of 5.04 and 5.20 for ResNet-20 and ResNet-110, respectively, compared to the previous results. We note that our results almost simulate the original backbone models, with classification accuracy differing from the backbone within 0.02%p. Furthermore, we show that the client's rotation key size generated and transmitted can be reduced from 105.6GB to 6.91GB for ResNet models during an MLaaS scenario. Finally, we show that our method can be combined with previous methods, providing flexibility for selecting batch sizes for inference.

1. Introduction

In a conventional Machine Learning as a Service (MLaaS) scenario, users upload their personal data to a cloud server, which then performs an inference phase on the data to provide the requested services. However, the users would be reluctant to upload their privacy-sensitive data such as medical information to the server, since they might not fully trust the server. Therefore, it is important for MLaaS providers to ensure that they protect the confidentiality and privacy of each individual during MLaaS.

Privacy-Preserving Machine Learning (PPML) provides a tool that can be utilized in this scenario. Fully Homomorphic Encryption (FHE) has been considered one of the most appropriate tools for PPMLs that achieve both strong security in the cryptographic sense and succinctness of communication. FHE supports computations on encrypted data without decrypting it, which enables MLaaS providers to perform inferences directly on the encrypted personal data of their clients.

Previous research has explored FHE-based solutions for secure inference, aiming to capture data privacy (Al Badawi et al., 2021; Boemer et al., 2019a;b; Chou et al., 2018; Dathathri et al., 2019; Gilad-Bachrach et al., 2016; Hesamifard et al., 2016; Kim & Guyot, 2023; Lee et al., 2022a; 2021; 2022b). However, only a few of them (Kim & Guyot, 2023; Lee et al., 2022a; 2021; 2022b) have specifically focused on deep Standard Convolutional Neural Networks (SCNN), such as the ResNet model, which is one of the most suitable models for MLaaS. Recently, the authors in (Lee et al., 2022a) proposed an efficient packing method and its related algorithms for homomorphic convolution evaluation using the RNS-CKKS scheme for ResNet models, and achieved notable results. They chose a strategy to pack a single data repeatedly into slots in a ciphertext for a single input inference, which enabled the simultaneous computation of multiple output channels by utilizing a Single-Instruction-Multiple-Data (SIMD) operation provided by CKKS scheme.

Although these methods achieved remarkably low latency for a single input inference in deep SCNN, they may not fulfill the demands of clients in various scenarios for MLaaS,

© 2024 IEEE. Personal use of this material is permitted. Permission from IEEE must be obtained for all other uses, in any current or future media, including reprinting/republishing this material for advertising or promotional purposes, creating new collective works, for resale or redistribution to servers or lists, or reuse of any copyrighted component of this work in other works.
DOI: 10.1109/TDSC.2024.3448406

¹Department of Mathematical Sciences, Seoul National University, Seoul, Republic of Korea ²CryptoLab Inc., Seoul, Republic of Korea. Correspondence to: Minsik Kang <kaiser351@snu.ac.kr>.

particularly when lots of inferences should be evaluated efficiently on the server. Indeed, the approaches in (Kim & Guyot, 2023; Lee et al., 2022a; 2021; 2022b) did not focus on providing optimized solutions to achieve efficiency in terms of throughput for *batch inference*, where a server can handle multiple inference calls from a client at once, covering a wide range of batch sizes. Batch inference is a highly promising scenario in MLaaS when the throughput of inference is more critical than response latency, such as in the field of user-customized recommendations (AmazonAWS, 2023).

Batch inference on CNN with FHE was first attempted in (Gilad-Bachrach et al., 2016) and since then, several works (Boemer et al., 2019a;b; Chou et al., 2018; Hesamifard et al., 2016) have made improvements in terms of accuracy and amortized timing. However, their constructions modified or redesigned the CNN models to be HE-friendly, which not only introduces inefficiency but also falls short of the MLaaS requirement of providing SCNNs. In (Lee et al., 2022a), the authors performed batch inference on ResNet models for multiple k images by assigning each individual image inference to a separate CPU thread, which required a total of k threads. We note that this approach may encounter challenges as the batch size for inference increases. These challenges arise from the memory overhead of managing large-size ciphertext and the limitations of resources available for implementation.

Our Contribution In this paper, we show that the limitations on batch inference can be overcome by distributing the tasks of batch inference not only to the CPU threads but also to the slots inside the ciphertext. We propose a more batch-friendly packing method, *Channel-By-Channel Packing* (CBC Packing), and an algorithm for convolution evaluation, *Channel-By-Channel Convolution* (CBC Conv), boosting up the throughput of secure ResNet inference by over $5\times$ compared to state-of-the-art previous works. The compactness of CBC Packing enables us to process a batch size that is not restricted by the number of threads, breaking the aforementioned throughput limitation. Also, CBC Packing improves Approximate ReLU (AppReLU), the most time-consuming part, by reducing the input ciphertexts to deal with and utilizing full-slot SIMD computation. The detailed improvements achieved by employing CBC methods for batch inference on CNN models are outlined below:

- Our proposed CBC Packing method reduces the number of AppReLU operations involving the most time-consuming operation, bootstrapping, from 19 to 5.75 per image.
- During the convolution operation, we eliminate the extra homomorphic multiplication operations between ci-

phertexts and auxiliary plaintexts, referred to as masks, using the CBC Conv method. Furthermore, we achieve a significant reduction in the number of heavy key-switching operations (KSOs) during convolutional layers and the fully-connected (FC64) layer, from 1,968 to 48.75 in an amortized sense.

- As the number of KSO for inference decreased, we correspondingly reduced the number of rotation keys required during inference from 265 to 45.
- We theoretically analyze that CBC method is compatible with the approach in (Lee et al., 2022a) for CNN inference and we propose a hybrid method that combines these two methods. This hybrid method can be applied across various batch sizes, offering increased flexibility to adjust the batch size according to the specific needs of clients in the MLaaS scenario.

We evaluate our improvements by implementing the standard ResNet-20/110 architectures utilizing the RNS-CKKS scheme provided by the HEaAN library¹. The obtained experimental results are as follows:

- We confirmed that our implementation achieves an amortized classification time of 15.76s for ResNet-20 and 87.60s for ResNet-110, respectively. These results are $5.04\times$ and $5.20\times$ faster than those of (Lee et al., 2022a), respectively. For ResNet-20, we achieve a $6.94\times$ reduction in amortized runtime for convolution with batch normalization and a $4.78\times$ reduction for AppReLU.
- Our inference implementations on ResNet-20 and ResNet-110 yields classification accuracies of 91.96% and 93.65%, respectively. These accuracies differ from the backbone models' accuracies of 91.98% and 93.63% within 0.02%, which implies our implementations almost simulate the backbone models.
- We found that the total memory of rotation keys for ResNet inference is about 6.91GB, which is $15.28\times$ smaller compared to the 105.6GB in (Lee et al., 2022c). We note that in the MLaaS scenario, a client has to transmit the required rotation keys to the server, which means that our method reduces the communication burden for the client.

2. Preliminaries

2.1. RNS-CKKS Fully Homomorphic Encryption

In our paper, all the operations inside the CNN architecture are in real arithmetic. CKKS (Cheon et al., 2017) naturally

¹Open-access on <https://heaan.it>

supports fixed-point arithmetic over real or complex numbers and can also handle multiple operations inside a single ciphertext in a SIMD manner. The ciphertexts in CKKS are elements of $\mathcal{R}_Q^2$, where $\mathcal{R}_Q = \mathbb{Z}_Q(X)/\langle X^N + 1 \rangle$ and N is a power-of-two integer. Each ciphertext encrypts a complex(or real) vector with $N/2$ slots as a message. The basic operations supported by CKKS are presented below. Here, $m_1, m_2 \in \mathbb{C}^{N/2}$ and $\odot$ denotes slot-wise multiplication.

$$\begin{aligned} \text{Dec}(\text{ctAdd}(\text{Enc}(m_1), \text{Enc}(m_2))) &\simeq m_1 + m_2 \\ \text{Dec}(\text{ptAdd}(\text{Enc}(m_1), m_2)) &\simeq m_1 + m_2 \\ \text{Dec}(\text{ctMult}(\text{Enc}(m_1), \text{Enc}(m_2))) &\simeq m_1 \odot m_2 \\ \text{Dec}(\text{ptMult}(\text{Enc}(m_1), m_2)) &\simeq m_1 \odot m_2 \\ \text{Dec}(\text{Rot}(\text{Enc}(m_1), t))[\text{idx}] &\simeq m_1[\text{idx} + t \bmod N/2] \end{aligned}$$

The message in the ciphertext contains some small errors, which accumulated during the homomorphic evaluation of circuits. CKKS manages this error accumulation by introducing a leveled modulus structure. In RNS-CKKS scheme (Cheon et al., 2018b), the ciphertext modulus Q is a product of word-size primes $Q = Q_L := \prod_{i=0}^L q_i$ so that the coefficients of the ciphertext polynomials can be decomposed under residue number system (RNS). Every RNS-CKKS ciphertext contains a *level* parameter l , which indicates the capacity of remaining homomorphic multiplications on the ciphertext, and each multiplication (ctMult or ptMult) consumes a single level. In general, the memory and computational cost depends on the ciphertext level.

After consuming all the levels of a ciphertext, one can recover the modulus by performing *bootstrapping* (BTS) (Cheon et al., 2018a) operation. If the data size is much smaller than the number of slots, we can pack the data repetitively in a single ciphertext, called *repetitive slot packing* (RS Packing). Specifically, we obtain a concatenated vector of the form $(\mathbf{x}|\mathbf{x}|\dots|\mathbf{x})$ from a vector $\mathbf{x}$ representing data and encrypt this concatenated vector in a full-slot ciphertext. We can execute some homomorphic operations more effectively using the RS Packing. In particular, we can exploit the RS Packing structure in linear transformations in BTS to reduce computational costs. This strategy is referred to as RS-BTS.

Key switching operation (KSO) is required for ciphertext-ciphertext multiplication (ctMult) and rotation (Rot). KSO accounts for most of the time during ctMult and Rot. These operations are much slower than addition and plaintext multiplication, which do not require KSOs. BTS contains multiple calls of ctMult and Rot and therefore requires many KSOs.

Public keys required for key switching in RNS-CKKS are large, and are generated in the largest modulus PQ . Here, P is called a *temporary modulus* (Gentry et al., 2012) and used in error control in KSOs. For a fixed security parameter,

PQ is also fixed, and the size of P is roughly inversely proportional to the *decomposition number* dnum value (Han & Ki, 2020). The total cost of the key-switching process is of $O((\text{dnum} + 1)L)$. Also, the size of the key-switching keys is proportional to dnum.

2.2. SISO convolution

Convolution involves multiplying an image vector with a strided kernel vector. To accomplish this on ciphertexts, we must appropriately pack the image and kernel into vectors in $\mathbb{C}^{N/2}$. In (Juvekar et al., 2018), the authors proposed a framework for secure CNN inference called Gazelle and suggested how to execute a *Single-Input-Single-Output* (SISO) convolution on HE schemes with SIMD properties. In the operation of SISO convolution, the encrypted image is rotated instead of the kernel, and $\sum_{i=0}^{i < k \times k} \text{ptMult}(\text{Rot}(\text{Enc}(\text{image}), \text{idx}_i), \text{kernel}_i)$ is calculated for single input channel and single output channel, where $k \times k$ denotes the size of the kernel.

2.3. Overall structure of ResNet models

Figure 1 represents the overall structure of our method to evaluate ResNet-20. For batch normalization (BN), we use the standard method of integrating BN into the preceding convolutional layer. We refer to the fused convolution as ConvBN. We call the residual blocks RB for regular residual block (Figure 1(a)) and DRB for downsampling residual block (Figure 1(b)), respectively. ResNet of higher depth shares the same structure with ResNet-20; the only difference is the number of RBs in each row in Figure 1(c). For ResNet-110, 15 RBs are added next to the end of each row.

2.4. Related Works

Previous works on Secure Inference The problem of secure inference can be solved via secure Multi-Party Computation (MPC) (Micali et al., 1987; Yao, 1986), homomorphic encryption (HE) (Gentry, 2009), or using Secure Processors (e.g., Intel SGX). To date, many MPC-based protocols or hybrid protocols combined with HE frameworks have been suggested for secure inference, but those commonly require clients to interact continuously with servers with higher online communication costs and rounds during inference. We refer to (Ng & Chow, 2023) for more details.

On the other hand, protocols solely using FHE involve only one communication round between a client and a server, offering less online communication costs compared to MPC-based protocols. CryptoNet (Gilad-Bachrach et al., 2016) is the pioneering study demonstrating the viability of secure inference using HE. Since then, numerous FHE-based protocols have been proposed as solutions for secure inference, employing a variety of FHE schemes (Brakerski et al., 2014;

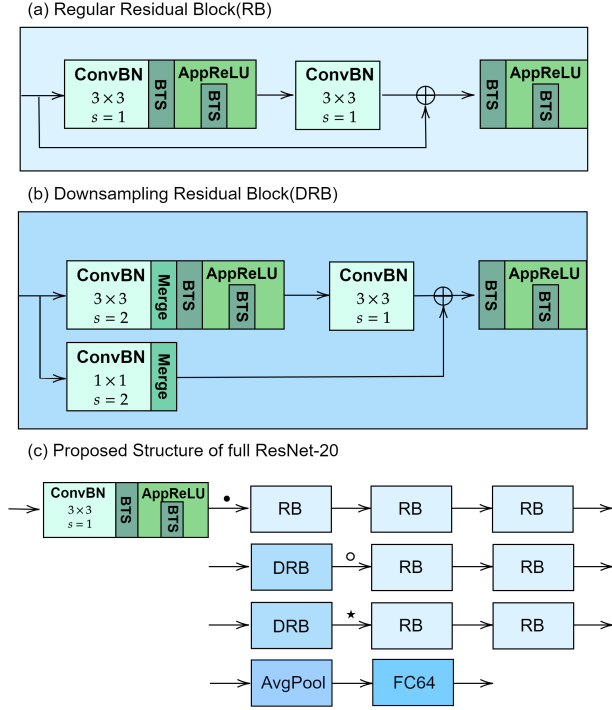


Figure 1. Overall structure of the proposed homomorphic evaluation of ResNet-20.

Cheon et al., 2017; Chillotti et al., 2020; Fan & Vercauteren, 2012). Brakerski/FanVercauteren (BFV) scheme is a word-wise FHE scheme that supports single instruction multiple data (SIMD) vectors by encoding multiple integers into a larger integer based on the Chinese Remainder Theorem. Leveraging this property, the BFV scheme has been widely used in protocols that are solely FHE-based (Al Badawi et al., 2021; Brutzkus et al., 2019; Lou et al., 2020) or combine FHE with MPC (Huang et al., 2022; Juvekar et al., 2018; Mishra et al., 2020; Ng & Chow, 2021). However, these BFV-based solutions require a quantization technique to convert the floating-point neural network parameters to the integer HE variables (Legiest et al., 2023), and solely BFV-based solutions like LoLa (Brutzkus et al., 2019) and Falcon (Lou et al., 2020) are limited to low-depth Neural Networks with FHE-friendly activation functions like the square function.

There is another line of work (Bourse et al., 2018; Lou & Jiang, 2019; Sanyal et al., 2018) proposed to evaluate discretized Neural Networks (NNs) using TFHE scheme (Chillotti et al., 2020). The TFHE scheme is a bit-wise HE that can handle arbitrary functions presented as boolean circuits. SHE (Lou & Jiang, 2019) gives the best result in this line of work, which translates the nonlinear ReLU and Max Pooling operations as Boolean operations to

achieve high inference accuracy. However, SHE also incurs significant inference latency and communication costs. For instance, predicting a single CIFAR-10 image in the ResNet-18 model takes 12,041 seconds and results in a ciphertext size of 160MB. Additionally, all TFHE-based works require appropriate discretization of network weights and inputs for efficient evaluation, which is not preferable in batch inference scenarios.

Recently, the CKKS FHE scheme (Cheon et al., 2017; 2018b) has been widely adopted in FHE-based inference since the CKKS is a word-wise HE supporting the fixed-point real number arithmetic in the SIMD manner and the computational time and precision of CKKS operations have been improved significantly in recent years (Bae et al., 2022; Cheon et al., 2023; Jung et al., 2021; Kim et al., 2022a; 2023). Compared to BGV or B/FV, the CKKS scheme is also advantageous in achieving high inference accuracy by employing high-precision polynomial approximation of non-linear activation functions like ReLU, rather than using square functions. Recently, (Lee et al., 2021) presented efficient polynomial approximation implementations for non-linear operations like ReLU and max-pooling functions, achieved by approximating them as compositions of low-degree polynomials (Cheon et al., 2020). For these reasons, the CKKS scheme has been regarded as the most suitable FHE scheme for implementing secure inference in deep standard NNs.

Inference on Deep Standard CNN There is a recent line of work (Lee et al., 2022a;b) which aimed to perform CNN inference on standard ResNet models. (Lee et al., 2022b) adapted the SISO convolution to additionally support strided convolutions of stride more than 1. In (Lee et al., 2022a), the authors named the packing method used in HEAR (Kim et al., 2022b) as *multiplexed packing*, and suggested an improved version of convolution with respect to this packing method to fill in the gaps with different channel data after strided convolutions. In addition, (Lee et al., 2022a) proposes *multiplexed parallel convolution* that utilizes SIMD with RS Packing structure of the ciphertext. However, there is still room to utilize SIMD more and it will be discussed throughout this paper.

Evaluation of non-linear layers Evaluating non-linear layers, such as the ReLU activation function or Max Pooling layers, is one of the most challenging tasks for secure inference. If one allows more than one communication round, the main bottleneck is high online communication costs for parties during inference. To mitigate the costs, (Rathee et al., 2021) implemented a library for communication-efficient secure inference, providing functionality for evaluating non-linear layers over real numbers based on a specialized 2-party computation protocol. Meanwhile, (Ng & Chow, 2021)

suggested a BFV-based solution with multiple communication rounds for secure inference. To achieve low latency with less online communication costs, they leveraged GPU parallelism in evaluating non-linear layers without approximation and proposed stochastic rounding and truncation layers to complement quantization-aware training.

On the other hand, non-interactive PPMLs based on the CKKS FHE scheme do not mandate clients to be online for evaluation of non-linear layers. In this case, a high-degree polynomial approximation is required to achieve the desired precision for the ReLU function during inference. There are prior works (Lee et al., 2022a; 2021; 2022b) which utilized a composition of small degree polynomials to approximate ReLU efficiently. The performance of homomorphic polynomial evaluation mainly depends on the number of iterations of ctMult involving the relinearization (Relin) operation, which is the special case of KSO. In (Lee et al., 2021), they utilized an odd baby-step giant-step (BSGS) algorithm to evaluate the approximate polynomial of a sign function. BSGS algorithm can be improved by adopting the lazy Relin, so-called lazy baby-step giant-step (lazy-BSGS) (Lee et al., 2022d). On the other hand, there is another technique to reduce the homomorphic evaluation error, called *imaginary-removing bootstrapping* (Lee et al., 2022a). This prevents the propagation of accumulated errors when evaluating the approximation of ReLU.

Reducing the required number of keys In (Lee et al., 2022c), the authors suggested *hierarchical Galois key management* in Neural Network inference to reduce the communication cost for the client to send the required rotation keys to the server. This method can significantly reduce the communication cost for public keys at the expense of increased computational costs on the server.

3. Channel-By-Channel Convolution on Fully Homomorphic Encryption

In this section, we introduce a new packing method and convolution algorithm based on it. We also analyze its algorithmic advantages when applied to the MLaaS scenario.

3.1. Channel-By-Channel Packing and Convolution

We suggest a compact packing method, *Channel-By-Channel Packing* (CBC Packing), where each ciphertext holds one channel data of multiple images. To utilize homomorphic SIMD with packed CKKS ciphertexts, previous naive approaches (Lee et al., 2022a;b) packed a 3 -channel $S \times S$ image in a single ciphertext of $N/2$ slots, $\left(\frac{N/2}{3 \times S \times S}\right)$ -times repeatedly (a.k.a. RS Packing). In batch inference scenarios, however, a ciphertext with $\left(\frac{N/2}{3 \times S \times S}\right)$ different input images makes more sense in utilizing SIMD. More-

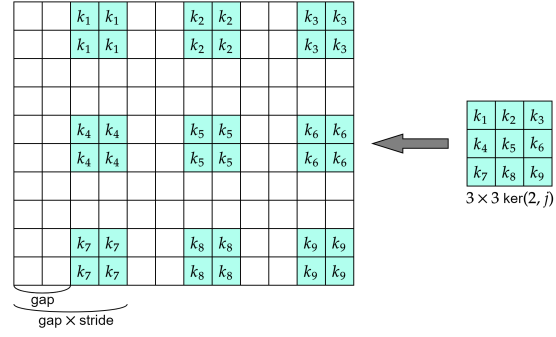


Figure 2. Kernel structure for Channel-By-Channel Convolution. (output channel=2, stride=2, gap=2, kernel size: 3×3)

over, our CBC Packing, which separates the images into k channels and packs each into k ciphertexts, reduces the number of redundant homomorphic rotations in the convolution phase. In summary, CBC Packing packs k -channel $\left(\frac{N/2}{S \times S}\right)$ images into k ciphertexts.

Definition 3.1 (Channel-By-Channel (CBC) Packing). Let $N/2$ be the number of slots in an RLWE ciphertext and $A_m = (a_{m,i,j,ch})_{0 \leq i,j < S, 1 \leq ch \leq k}$ be the m -th square image of size $S \times S$ with k channels for simplicity. For $M := \left\lceil \frac{N}{2S^2} \right\rceil$, note that a ciphertext can pack M number of a single channel images.

Then, CBC Packing packs images $A_1, \dots, A_M$ as vectors $P_{ch} = (p_{ch,\ell})_{0 \leq \ell < N/2}$ for $ch = 1, \dots, k$;

$$p_{ch,\ell} = a_{q_\ell+1, q_r, \ell-S^2 \cdot q_\ell - S \cdot q_r, ch},$$

where $q_\ell = \left\lfloor \frac{\ell}{S^2} \right\rfloor$ and $q_r = \left\lfloor \frac{\ell}{S} \right\rfloor$.

This is also depicted in Figure 3 (b). Here, pixels of the same color represent ciphertexts encrypting image data for the same channel. Each pixel in Figure 3 (b) represents an image of a single channel that is packed into a message vector in a raster-scan fashion. Our CBC Packing differs from gap packing in (Lee et al., 2022b), which merely packs multiple channel data of a single image per ciphertext.

We then propose an efficient homomorphic convolution algorithm, denoted as CBC Conv (Algorithm 1), that achieves high-throughput results for batch inference. Our CBC method makes intensive use of all the slots in the ciphertext, taking full advantage of the SIMD operations. Furthermore, the CBC method can minimize the number of heavy KSOs, such as rotations for the slots within the ciphertext, while maintaining the CBC Packing structure during convolution operation.

Algorithm 1 CBC ConvBN

Input: $\text{imagesize} = \text{img}$, $\text{gap} = g$, $\text{input_channel} = \text{in_ch}$, $\text{output_channel} = \text{out_ch}$, ciphertext bundle = ct , 3-dim output_kernel_bundle = $\text{kernel} = \{\text{ker}_k(i, j)\}$ (k is the SISO Conv index), corresponding summand vector = sum .

Output: ciphertext vector ct_{res} having convolution result.

procedure CBCConvBN(img , g , s , in_ch , out_ch , ct , kernel , sum)

$\text{ker_size} \leftarrow \text{size}(\text{ker}_k(i, j))$

$n \leftarrow \sqrt{\text{ker_size}}$ (kernel size : $n \times n$)

for $0 \leq i \leq \text{in_ch} - 1$; $0 \leq j \leq n - 1$; $0 \leq k \leq n - 1$
 $\text{ct}_{\text{rot}}^{i, n*j+k} \leftarrow \text{Rot}(\text{ct}_i, -(j-1)*\text{img}*g - (k-1)*g)$

end for

for $0 \leq i \leq \text{out_ch} - 1$

for $0 \leq j \leq \text{in_ch} - 1$

$\text{ct}_{\text{temp1}} \leftarrow \text{multWithoutRescale}(\text{ct}_{\text{rot}}^{i,0}, \text{ker}_0(i, j))$

for $1 \leq k \leq n^2 - 1$

$\text{ct}_{\text{temp2}} \leftarrow \text{multWithoutRescale}(\text{ct}_{\text{rot}}^{i,k},$

$\text{ker}_k(i, j))$

$\text{ct}_{\text{temp1}} \leftarrow \text{ctAdd}(\text{ct}_{\text{temp1}}, \text{ct}_{\text{temp2}})$

end for

if $j = 0$

$\text{ct}_{\text{res},i} \leftarrow \text{ct}_{\text{temp1}}$

else

$\text{ct}_{\text{res},i} \leftarrow \text{ctAdd}(\text{ct}_{\text{temp1}}, \text{ct}_{\text{res},i})$

end for

$\text{ct}_{\text{res},i} \leftarrow \text{ReScale}(\text{ct}_{\text{res},i})$

end for

$\text{ct}_{\text{res}} \leftarrow \text{ptAdd}(\text{ct}_{\text{res}}, \text{sum})$

return ct_{res}

end procedure

CBC Conv uses special forms of kernels to reduce the redundant rotations. Here, we explain the plaintext packing structure of a kernel $\text{ker}(i, j)$ for j -th input channel and i -th output channel. This is illustrated in Figure 2, with zero slots represented as white pixels. We now assume that the kernel has a size of 3×3 for simplicity. For a gap (of the input ciphertext) g and a stride s , consider a $3gs \times 3gs$ matrix K of $\text{ker}(i, j)$. First, fill out the nine $gs \times gs$ blocks of the matrix with the nine values of $\text{ker}(i, j)$, respectively. Except for the i -th ($i \leq s^2$) $g \times g$ sub-block, erase the other eight sub-blocks into zero. Finally, the output matrix of $\text{ker}(i, j)$ is also encoded in a raster-scan fashion to fit with the CBC packed inputs. One can encode a single kernel $\text{ker}(i, j)$ into 3×3 number of plaintexts that correspond to the nine SISO kernels (Juvekar et al., 2018) for a 3×3 kernel, respectively.

With the encoded kernel above, CBC Conv can be understood as “a channel-separated masked SISO convolution”. For each output channel, CBC Conv for convolution with a stride of 1 performs SISO convolutions (Juvekar et al., 2018) over input channels and summation, which can be done in parallel. Here, no action other than the summation of ciphertexts is required to collect data over input channels. The procedure of CBC Conv performed in the ResNet inference is described in Figure 3 (b) and Figure 4 (b). CBC Conv also supports the strided convolution (*i.e.* $s \geq 2$), as shown in Figure 4 (b). As can be seen in the figure, an image ciphertext goes through a homomorphic rotation and is then multiplied by the respective kernel plaintext, which is repeated for each (i, j) -th kernel. Then, we sum up the results over i and j to obtain an output of SISO convolution. Next, we again sum up the SISO output ciphertexts to collect the values from multiple input channels, as illustrated in the second row. The operations do not differ from the case $s = 1$ except that the kernels are generated considering the change of gaps. Here, the gap denotes the distance between slots from the same image within the ciphertext. In Figure 4 (b), the case of the gap increasing from 2 to 4 is illustrated. Another difference is that the Merge (see Algorithm 2) step is added, which merges the outputs from other images, making the ciphertext compact again.

3.2. Algorithmic Improvements via CBC method

Our CBC method approach achieves algorithmic improvements over MP Conv in the homomorphic convolution operation for batch inference on multiple images.

Compact Packing Multiplexed-Parallel Packing (MP Packing) method in (Kim et al., 2022b; Lee et al., 2022a) exploits RS packing to pack one $3 \times 32 \times 32$ CIFAR-10/100 image data into one ciphertext of 2^{15} slots, as shown in Figure 3 (a). RS-BTS is used in (Lee et al., 2022a) to reduce the runtime of bootstrapping in inference.

Algorithm 2 Merge

Input: ciphertext vector = $\mathbf{ct}$, imagesize = img , stride = s and gap = g

Output: rotated sum ciphertext $\mathbf{ct}_{sum}$

Note that $\mathbf{ct}_{res}$ is an initialized ciphertext vector and $\mathbf{ct}_{sum}$ is an initialized ciphertext which is an encryption of zero message.

procedure Merge($\mathbf{ct}$, img , s , g)

for $0 \leq i \leq s-1; 0 \leq j \leq s-1$

$\mathbf{ct}_{res}^{s*i+j} \leftarrow \text{Rot}(\mathbf{ct}^{s*i+j}, -(img * g * i) - j * g)$

$\mathbf{ct}_{sum} \leftarrow \text{ctAdd}(\mathbf{ct}_{res}^{s*i+j}, \mathbf{ct}_{sum})$

end for

return $\mathbf{ct}_{sum}$

end procedure

However, using MP Packing when packing multiple image data into a ciphertext rather than one into a ciphertext may result in inefficiency in terms of throughput. Intuitively, it is more efficient in an amortized sense to pack as much data as possible into the slots rather than repeatedly filling the remaining slots in the ciphertext with RS Packing. We also note that some slots in the ciphertext are wasted when utilizing MP Packing due to the number of input channels. The image data such as CIFAR-10/100 typically has 3 RGB color channels, which is not a power-of-two. Since current FHE schemes such as CKKS primarily support ciphertexts with a power-of-two number of slots, it is necessary to fill in the rest of the remaining slots with zero, illustrated by the white pixels in Figure 3 (a).

On the other hand, we utilize the CBC Packing method to pack 32 CIFAR-10/100 images into 3 ciphertexts by getting rid of those zero channels as shown in Figure 3 (b). After performing one stride 2 convolution during inference, we can pack four times as many images simultaneously to preserve ciphertexts' CBC structure. For batch inference on the ResNet-20 model, we compactly pack 512 images into 3×16 ciphertexts. The ciphertexts are processed in parallel during inference, keeping the channels of the images separated until the FC64 layer. The overall structure is illustrated in Figure 5.

Reduced number of KSO and ptMult Compared to the MP Conv in (Lee et al., 2022a), the CBC Conv, which employs CBC Packing for homomorphic convolution, offers advantages in terms of both compactness of packing and reduction of KSOs and ptMults in each convolution operation during inference.

Specifically, each MP Conv involves the Rotate-and-Sum (RotSum) process to combine values within a single ciphertext, as shown in Figure 4(a). During the RotSum, the slots that do not contribute to the sum are filled with dummy

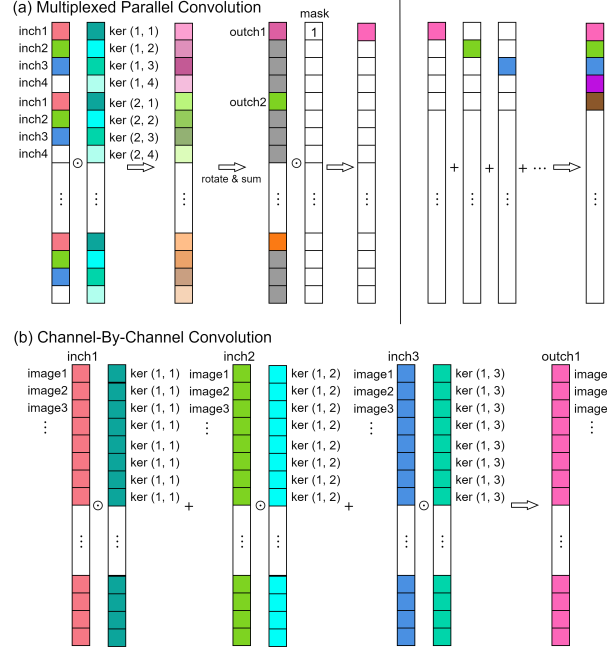


Figure 3. Comparison between Channel-By-Channel Convolution and Multiplexed Parallel Convolution. (input channel=3, stride=1)

garbage values, represented by the grayed-out pixels in Figure 3. Consequently, MP Conv requires a Gather step, which involves ptMult with masked plaintexts, rotation, and addition, to assemble sparse valid data from the same or different ciphertexts into a more compact single ciphertext in an RS manner. We note that MP Conv necessitates the Gather step for compact packing of output ciphertexts for every homomorphic convolution in CNN inference.

On the other hand, the proposed CBC Conv does not generate garbage values during convolutions, eliminating the need for gathering ciphertexts during stride 1 convolutions, as depicted in Figure 3(b). The whole procedure combined with BN is illustrated in Algorithm 1.

In the case of strided convolutions, the Gather step in CBC Conv is replaced by the Merge step, which consists of simple ctAdds and Rot, without further ptMult with masked plaintexts, as depicted in Figure 4. We point out that this simple Merge step is used only for convolutions with a stride of 2 in ResNet inference.

3.3. Advantages in an MLaaS Scenario

In summary, CBC Conv performs a more compact and simultaneous evaluation for a batch of multiple images, effec-

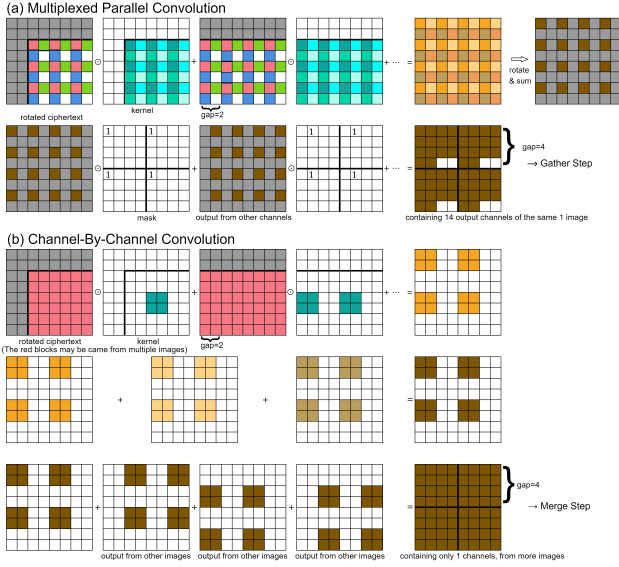


Figure 4. Comparison between Channel-By-Channel Convolution and Multiplexed Parallel Convolution. (input channel=3, output channel=14, stride=2)

Table 1. The amortized number of key-switching operations of the proposed architecture using Channel-by-Channel Convolution compared to Multiplexed Parallel Convolution (Lee et al., 2022a) in ResNet-20.

Component	proposed	MP Conv
Convolution	48	1,902
Avgpool	0.75	6
FC Layer	0	60
Total	48.75	1,968

tively excluding additional rotations by maximizing SIMD operations. These optimizations lead to the advantage of eliminating the ptMult with additional masked plaintexts and significantly reducing amortized KSOs, from 1,968 in (Lee et al., 2022a) to 48.75, as shown in Table 1. As a result, CBC Conv provides the following benefits in an MLaaS scenario.

- Due to the compactness of our CBC Packing method, we can reduce the number of BTSs required per image during the CNN inference. We also significantly reduce the number of heavy KSOs for homomorphic evaluation of convolution in batch inference. The decrease in operations results in $5.04\times$ and $5.20\times$ reduction in amortized runtime for ResNet-20 and ResNet-110, respectively.
- In the CKKS scheme, homomorphic operations such

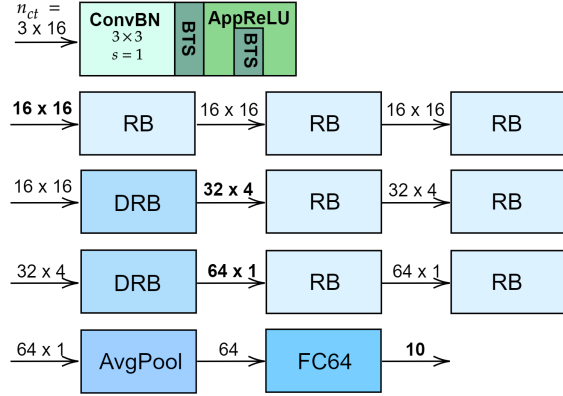


Figure 5. The number of ciphertexts n_{ct} between the components of ResNet-20 for batch inference of 512 images.

as KSO and ptMult inevitably introduce additional errors to the original message. We have observed that our CBC Conv approach effectively minimizes the degradation of classification accuracy in ResNet inference. As shown in Table 6 of Section 6.2, our method achieves nearly identical accuracy when compared to the backbone model.

- In an MLaaS system of the client-server model using FHE schemes, the server needs to ask clients to generate and send all necessary rotation keys for KSO operations using their secret keys. The CBC Conv approach not only achieves ciphertext compactness but also leads to reduced public key size and lower communication costs. Specifically, the required number of (public) rotation keys decreases to 45, compared to 265 as can be found in (Lee et al., 2022a;c; Snu-ccl, 2022).

4. Hybrid Method for Flexible Batch Size

Although our CBC method achieves high throughput during batch inference, it lacks flexibility in the choice of batch size. For ResNet-20/110, the CBC method requires 512 images at a time to perform inference, which incurs additional latency when we try to perform inference on fewer images than 512. In other words, one can find that there is a trade-off between latency and throughput for an inference structure, depending on the batch size. In this section, we suggest a solution that controls the trade-offs between batch size, throughput, and latency. Let M Packing and M Conv stand for the multiplexed packing and the corresponding multiplexed convolution without RS Packing, respectively. We note that, M Pack with RS Packing is just MP Packing.

CBC Conv does not conflict with the M Conv (Lee et al.,

2022a), so they can be used together in a CNN model. This is because each CBC Conv and M Conv separately affects only a single convolution.

The choice between CBC Conv and M Conv at each convolution stage is determined by the number of ciphertexts in each step and the shape of packed images in a ciphertext. One can also consider a single convolution, a mixed version of CBC Conv and M Conv. This mixed convolution should be used for some of the batch sizes which will be explained later.

In this paper, ResNet-20 will be considered as a baseline for the explanation of the hybrid method. We suggest a hierarchy of packing methods to utilize the aforementioned trade-off effectively. From the batch size of 512 images of CBC Packing to the batch size of 1 image of M Packing, we pack the intermediate batch size of images by considering the following hierarchy;

CBC Packing $\rightarrow$ M Packing $\rightarrow$ RS Packing.

We note that the 48 ciphertexts, containing 512 images, in CBC Packing consists of 16 bundles of 3 channels. Table 2 illustrates the number of ciphertexts and required convolutions in each point ($\bullet$, $\circ$, $\star$) in Figure 1.

There are some *rules* to follow for maximal efficiency;

1. Before BTS, Merge or Gather to make ciphertexts fully compactly packed.
2. If the number of ciphertexts after compact packing is greater or equal to 2, the compact packing must separate channels into distinct ciphertexts instead of separate images.
3. If there exist zero slots due to the gaps even after Merge or Gather, use RS Packing to accelerate BTS.

Table 2. The number of ciphertexts in each step for various batch sizes. Block 1, 2, and 3 stand for the marked flow in Figure 1 (c).

Batch	Block1 ($\bullet$)	Block2 ($\circ$)	Block3 ($\star$)	BTS	amor. BTS
512	256	128	64	5888	11.5
256	128	64	32	2944	11.5
128	64	32	16	1472	11.5
64	32	16	8	736	11.5
32	16	8	4	368	11.5
16	8	4	2	184	11.5
8	4	2	1	92	11.5
4	2	1	1	52	13
2	1	1	1	38	19
1	1	1	1	38	38

Batch sizes 128 and 32 First, we should reduce the batch size from 512 by reducing the distinct 16 bundles of initial input images. In the case of batch size 128, there are two stride 2 convolutions, so we have to decide which data should be merged at each DRB. For the first DRB, we should use Merge following the rule 1 and 2 above. For the second DRB, the 4 different channels should be merged into a ciphertext by Gather following the rule 1 and 2 again. Therefore, there is an M Conv only after the last downsampling, and all the rest of the convolutions are CBC Conv. Similarly, in the case of batch size 32, there needs two Gather, and all the convolutions are M Conv after the first downsampling convolution of DRB. In Table 2, the green stands for that there needs CBC Conv right after that point, and the blue stands for that there needs M Conv right after that point.

Batch size less than 16 For the batch size smaller than 16, M Packing should be used from right after the first convolution following the rule 1 and 2. One can check that the ciphertexts in the yellow colored in Table 2 are packed by RS Packing following the rule 3. Then M Conv and RS-BTS can be utilized. One can also check them from the yellow color in Table 2 that stands for RS Packing.

Batch sizes 256, 64, and 16 For the batch sizes 256, 64, and 16, there needs to be a mixed version of convolution between CBC Conv and M Conv to follow the rule 1 and 2. Consider the ciphertexts at the red box steps in Table 2. Right before we compactly pack ciphertexts into the red-boxed number of ciphertexts, the number of channels is twice bigger than the number of ciphertexts. Since Merge repacks 4 ciphertexts into a ciphertext in the case of ResNet-20, we need a mixed version of the compact packing method between Merge and Gather. First, the compact packing method repacks 2 ciphertexts with Merge. After that, it repacks 2 output ciphertexts of Merge with Gather. So there also needs to be a mixed version of convolution between CBC Conv and M Conv, as we mentioned above.

Figure 6 shows the whole summary of the packing procedure to utilize the hybrid method. We present the amortized runtime per image for various batch sizes in Table 3 by estimating the dominant BTS time based on the experimental amortized time for CBC Conv for 512 batch size.

5. Fine-Tuning for Homomorphic Inference Evaluation

In this section, we present ad-hoc optimizations which can be applied when we implement ResNet-20/110 from existing techniques. There are many choices to implement the techniques in ResNet, so we suggest good choices to improve the overall performances including amortized runtime and accuracy. From now on, we focus on the newly pro-

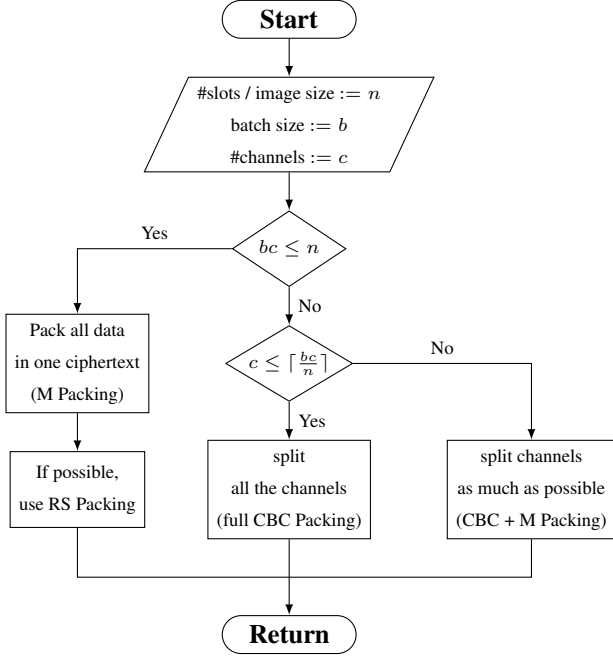


Figure 6. Flow chart for determining the packing structure for the hybrid method.

posed CBC Conv structure in ResNet-20/110 with a batch size of 512.

5.1. Lazy Rescaling

The computational bottleneck of the SISO convolution method (Juvekar et al., 2018) comes from the numerous ptMults between the input ciphertexts and the kernel plaintexts. Inside the procedure of ptMult, rescaling accounts for most of the runtime and induces additional errors. In general, when we need to sum up the outputs of ptMult, it is possible to postpone the rescaling until the summation, which is called the lazy rescaling technique. The rescaling is performed only once after the summation to reduce the error and computational costs. We exploit the arithmetic structure in convolution and FC64 layer to adopt the lazy rescaling.

5.2. Ciphertext Level Management

We can process the same homomorphic operations with less computational cost under a lower modulus. Since the cost of bringing levels down is negligible in RNS-CKKS, we can optimize the computational complexity by maintaining the ciphertext level as low as possible if we can adjust the level at which the operations are executed. We can manage levels in three parts; ConvBN, AvgPool, and FC64.

Our selection of the parameters in HEaAN requires at least 3 remaining levels before BTS, and if we have more level budget than needed after BTS, we can adjust the level before

Table 3. Estimated amortized runtime for ResNet-20 assuming our 40 thread environment.

Batch	Total BTS batch	Amor. BTS batch	amor. time (sec.)
512	170	0.3320	15.7576
256	92	0.3594	17.0553
128	52	0.4063	19.2799
64	38	0.5938	28.1783
32	38	1.1875	56.3567
16	38	2.3750	112.7133
8	38	4.7500	225.4266
4	38	9.5000	450.8532
2	38	57.2235	901.7065
1	38	114.4471	1803.4130

the heavy computation to optimize the computational cost. The followings are parts of our methodology where we can deploy level optimization:

- (1) The first ConvBN: Our method for ConvBN costs only 1 level, and the ReLU layers require 4 levels for the input: 1 level for the imaginary-removing BTS and 3 levels before BTS. Originally, the plaintexts are encoded to have 12 levels by default. So, before going through the first convolution, we decrease the level of the encrypted image vectors from 12 down to 5.
- (2) Other ConvBNs: The second through the last ConvBNs receive outputs of ReLU as inputs. After the second bootstrapping of each ReLU layer, the remaining polynomial evaluation for ReLU consumes 6 levels. Since we have an extra 1 level not being used, we adjust the level after the BTS from 12 to 11 so that the level is consumed tightly, as shown in Figure 7.
- (3) AvgPool and FC64: In average pooling, we calculate the average of 8×8 data inside a ciphertext. Rather than calling ptMult for the factor $1/64$ and consuming 1 level, we merge the multiplication to the parameters of the FC64 layer. AvgPool and FC64 layer cost only 1 level. So we reduce the level after the last BTS by 5, from 12 down to 7.

5.3. ReLU Activation function

To evaluate the ReLU activation function homomorphically, we utilize an approximate polynomial for ReLU (AppReLU) by employing a composition of minimax approximate polynomials, following the methodology described in (Lee et al., 2022a; 2021; 2022b). More precisely, we exploit three odd polynomials p_1, p_2 and p_3 of degree 15, 15, and 27, respectively, to evaluate AppReLU as follows:

$$\text{AppReLU}(x) = x \cdot \frac{1 + \text{sgn}(x)}{2}, \text{ where}$$

Table 4. Comparison of total and amortized inference time for ResNet-20/110. Numbers in * marks are calculated from total time and percentage data provided in (Lee et al., 2022a).

Model	ResNet-20								ResNet-110					
	(Lee et al., 2022b) (1 image, 64 threads)		(Lee et al., 2022a) (1 image, single thread)		(Lee et al., 2022a) (50 images, 50 threads)		proposed (512 images, 40 threads)		(Lee et al., 2022a) (1 image, single thread)		(Lee et al., 2022a) (50 images, 50 threads)		proposed (512 images, 40 threads)	
	runtime	amor.	runtime	amor.	runtime	amor.	runtime	amor.	runtime	amor.	-	amor.	runtime	amor.
ConvBN	-	-	351s	-	614.06s*	12.28s*	906.84s	1.77s	1,860s	-	3,190s*	63.80s*	5237.48s	10.25s
AppReLU	-	-	1,908s	-	3,337.95s*	66.76s*	7,154s	13.97s	11,411s	-	19,569s*	391.39s*	39,593s	77.33s
AvgPool	-	-	2s	-	3.50s*	0.070s*	0.91s	0.002s	2s	-	3.43s*	0.069s*	0.50s	0.001s
FC Layer	-	-	10s	-	17.49s*	0.350s*	6.00s	0.012s	10s	-	17.15s*	0.343s*	5.77s	0.011s
Total	10,602s	-	2,271s	-	3,973s	79.46s	8,067.90s	15.76s	13,282s	-	22,778s	455.56s	44,850.24s	87.60s

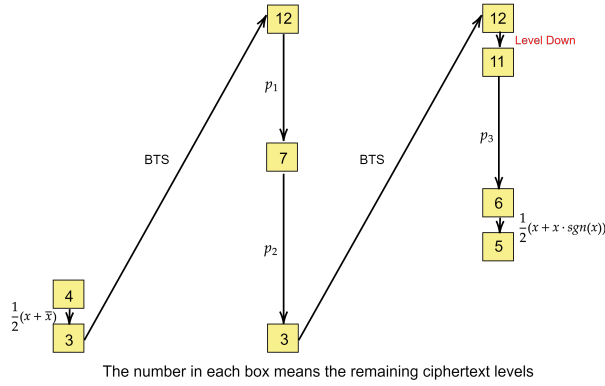


Figure 7. The whole procedure of our method during evaluation AppReLU. We maintain the ciphertext level to 5 before the residual block to optimally manage the ciphertext level. Our method reduces the number of ReLin evaluating AppReLU from 26 to 22 compared to previous work.

$$\text{sgn}(x) \approx (p_3 \circ p_2 \circ p_1)(x).$$

Using the lazy-BSGS algorithm (OddEvalPoly) (Lee et al., 2022d) to homomorphically evaluate the AppReLU in our model, we reduce the total number of ReLins. When implementing the lazy-BSGS algorithm, there is a trade-off between consuming depth and the number of ReLin according to the choice of k, l .

We select (k, l) as $(4, 2)$, $(2, 3)$, and $(4, 3)$ for p_1 , p_2 , and p_3 , respectively. By taking another choice of k, l for degree 15 polynomials, we can reduce the number of ReLin without consuming more depth in the overall AppReLU algorithm. In conclusion, the total number of ReLin required in our method is 22, which reduces 26 to 22 compared to previous work. Figure 7 presents the whole procedure of our method to evaluate AppReLU homomorphically.

Algorithm 3 AppReLU

Input: A ciphertext ct of m and polynomials p_1, p_2, p_3 with coefficients $\{a_i\}, \{b_i\}, \{c_i\}$ of degree n_1, n_2, n_3 .
Output: A ciphertext encrypting $\text{AppReLU}(m)$.
 Let k_i be an even integer and l_i is the smallest positive integer satisfying $2^{l_i} k_i > n_i$.

```

procedure AppReLU( $ct, \{a_i\}, \{b_i\}, \{c_i\}$ )
   $ct_{im\_bts} \leftarrow \text{imaginary-removing-BTS}(ct)$ 
   $ct_1 \leftarrow \text{OddEvalPoly}(ct_{im\_bts}, \{a_i\}, k_1, l_1)$ 
   $ct_2 \leftarrow \text{OddEvalPoly}(ct_1, \{b_i\}, k_2, l_2)$ 
   $ct_{bts} \leftarrow \text{BTS}(ct_2)$ 
   $ct_3 \leftarrow \text{OddEvalPoly}(ct_{bts}, \{c_i\}, k_3, l_3)$ 
   $ct_{res} \leftarrow ct_{\text{Mult}}(ct, pt_{\text{Mult}}(pt_{\text{Add}}(1, ct_3), 0.5))$ 
return  $ct_{res}$ 
end procedure

```

6. Implementation

6.1. Environment

We implement the ResNet-20/110 model on the following computing environment: 2 × Intel(R) Xeon(R) Gold 6248 CPU at 2.50GHz with 20 cores (total 40 cores) and 1TB RAM. We conduct our experiments on the HEaAN docker image loaded on Ubuntu-20.04. We execute inference with parameters pre-trained on the CIFAR-10 dataset which is comprised of 50,000 images for training and 10,000 images for testing (Krizhevsky, 2009).

Library HEaAN (CryptoLab, ver. September 2022) is a homomorphic encryption library supporting the (RNS-)CKKS scheme and its BTS. Since the implementation environment is slightly different from the previous works (Lee et al., 2022a;b), we provide a rough comparison of the latency for basic operations in RNS-CKKS between HEaAN

and SEAL library in Table 5.²³ SEAL library supports the ‘full dnum’ scheme. For example, in the work of (Lee et al., 2022a), the parameter choice was 51-bit base prime and 51-bit special modulus, and 46-bit multiplication primes. The total number of primes is 33, and dnum = 32.

Table 5. Evaluation time of operations in HEaAN and SEAL Library

	Rot	ptMult	ptMult w/o Rescale	Rescale	BTS
HEaAN	85ms	21ms	2ms	19ms	23s
SEAL	69ms	17ms	2ms	15ms	140s*

Parameters We use a parameter preset **FGb** provided in HEaAN, which has $N = 2^{16}$ polynomial degree and secret key hamming weight 192. FGb uses dnum = 5, with special modulus P consisting of 5 primes and Q consisting of 25 primes. The total level of ciphertext provided by **FGb** parameters is thus 24, of which 15 are reserved to BTS, and users can use the maximum multiplicative depth 9, from level 12 to level 3. **FGb** parameters have a 58-bit base prime and 42-bit primes for multiplication. The total modulus size, including the special primes, is 1555-bit, with 128-bit security. A key-switching key size at maximum level is about 157.3MB for this parameter.

6.2. Experimental Results

This section presents the results of ResNet-20/110 implementation performance according to our architecture using full CBC Conv in contrast to previous works. In (Lee et al., 2022a), the authors first implemented ResNet models by using one CPU thread with one image and expanded the models to test 50 images with 50 CPU threads by allocating one CPU thread per image. On the other hand, we perform ResNet models to test 512 images at once using 40 CPU threads.

CBC Runtime Our experimental results demonstrate improvements in terms of throughput with the CBC method on ResNet-20 and 110, as shown in Table 4. The implementation of the CBC method with 512 images takes 15.76s for one image classification in an amortized sense, which is a $5.02\times$ improvement from 79.46s in (Lee et al., 2022a). The amortized runtime in convolution layers is 1.79s, which is a $6.94\times$ improvement compared to previous work. We also

²³Specification details: Intel(R) Xeon(R) Silver 4114 CPU at 2.20GHz, single-thread bound to a single core; HEaAN (CryptoLab, ver. September 2022), FGb parameter ($N = 2^{16}$, base prime 58 bit, multiplication prime 42 bit); SEAL v4.1.1 (SEAL, January 2023), custom parameter ($N = 2^{16}$, base prime 51 bit, multiplication prime 46 bit); evaluated at level 5 with modulus size 268-bit for HEaAN and 281-bit for SEAL except for BTS.

³SEAL BTS time is excerpted from (Lee et al., 2022a).

accelerate AppReLU, AvgPool, and FC Layer procedures in our model by $4.78\times$, $39.22\times$, and $29.83\times$, respectively. We also implement ResNet-110, which results in a $5.20\times$ reduction in amortized runtime. The source code is available ⁴.

Table 6. Classification accuracy of ResNet20/110 on CIFAR-10.

	model	#test images	backbone	obtained
proposed	ResNet-20	10,000	91.98%	91.96%
	ResNet-110	10,000	93.63%	93.65%
(Lee et al., 2022a)	ResNet-20	10,000	91.52%	91.31%
	ResNet-110	10,000	93.5%	92.95%

Classification Accuracy We present our classification accuracies for ResNet-20/110 as in Table 6. There are only under 0.02%p accuracy differences, both in the classification of ResNet-20 and ResNet-110, compared to backbone accuracies. We note that an accuracy degradation of 0.21%p and 0.55%p occurred when implementing ResNet-20 and ResNet-110, respectively in (Lee et al., 2022a). By reducing the additional KSOs, our method achieved a genuine simulation of the backbone model with negligible accuracy loss.

Memory Management Our implementation for ResNet models has RAM usage by 370.28GB at maximum, compared to the 384GB and 512GB for 1 image and 50 images, respectively, in (Snu-ccl, 2022). We note that most of our RAM usage comes from the storage of plaintexts that encodes pre-trained parameters. Thanks to our efficient CBC Packing method, the memory usage for rotation keys is 6.91GB, which is relatively small in the whole RAM usage. We used the (left) rotation key set of size 45: $\{\pm 1, \pm 2, 3, \pm 4, 5, 6, 7, \pm 8, \pm 16, \pm 24, \pm 32, 33, \pm 64, 66, 96, \pm 128, 160, 192, 224, \pm 256, \pm 512, \pm 768, \pm 1024, 2048, 3072, 4096, 5120, 6144, 7168, \pm 8192, 16384\}$. This is a significant improvement compared to previous works, where the homomorphic ResNet architecture of (Lee et al., 2022a) required 265 rotation keys, adding up to 105.6GB when using dnum = 12, simulated in Lattigo library (EPFL-LDS, 2022, v3; Lee et al., 2022c).

Communication Cost Our method has another advantage of reduced communication costs. For 512 images, our method packs all the images into 48 ciphertexts at level 5, having a total size of 0.29GB. In the previous method of (Lee et al., 2022a), each ciphertext includes a single image, so 512 ciphertexts at level 31 needs to be sent, which adds up to 15.25GB. Furthermore, for the rotation keys, the hierarchical Galois key management method (Lee et al., 2022c) can also be applied to reduce the communication cost.

⁴<https://github.com/altaria1/HEaAN-CBC-ResNet>

7. Conclusions

We have constructed a new packing method that effectively leverages the “batch” inference scenario. Our simulation results demonstrate significant improvements in amortized time. Our packing method can coexist with the MP Packing, enabling us to explore various batch scenarios. So, we provide more flexibility in choosing between latency and throughput trade-offs via theoretical analysis. We also increase the practicality of our ResNet models by effectively minimizing error-inducing operations, achieving almost identical accuracy as the backbone models and downsizing key size for communication. We expect that our CBC method can be applied to other standard CNN networks, which results in increased efficacy considering their number of downsampling layers.

References

- Al Badawi, A., Jin, C., Lin, J., Mun, C. F., Jie, S. J., Tan, B. H. M., Nan, X., Aung, K. M. M., and Chandrasekhar, V. R. Towards the alexnet moment for homomorphic encryption: HCNN, the first homomorphic cnn on encrypted data with gpus. *IEEE Transactions on Emerging Topics in Computing*, 9(3):1330–1343, 2021. doi: 10.1109/TETC.2020.3014636.
- AmazonAWS. Amazon personalize - developer guide; Getting batch recommendations and user segments. <https://docs.aws.amazon.com/personalize/latest/dg/getting-started.html>, 2023.
- Bae, Y., Cheon, J. H., Cho, W., Kim, J., and Kim, T. Meta-bts: Bootstrapping precision beyond the limit. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*, pp. 223–234, 2022.
- Boemer, F., Costache, A., Cammarota, R., and Wierzynski, C. ngraph-he2: A high-throughput framework for neural network inference on encrypted data. In *Proceedings of the 7th ACM Workshop on Encrypted Computing & Applied Homomorphic Cryptography*, pp. 45–56, 2019a.
- Boemer, F., Lao, Y., Cammarota, R., and Wierzynski, C. ngraph-he: a graph compiler for deep learning on homomorphically encrypted data. In *Proceedings of the 16th ACM international conference on computing frontiers*, pp. 3–13, 2019b.
- Bourse, F., Minelli, M., Minihold, M., and Paillier, P. Fast homomorphic evaluation of deep discretized neural networks. In *Advances in Cryptology—CRYPTO 2018: 38th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 19–23, 2018, Proceedings, Part III 38*, pp. 483–512. Springer, 2018.
- Brakerski, Z., Gentry, C., and Vaikuntanathan, V. (leveled) fully homomorphic encryption without bootstrapping. *ACM Transactions on Computation Theory (TOCT)*, 6(3):1–36, 2014.
- Brutzkus, A., Gilad-Bachrach, R., and Elisha, O. Low latency privacy preserving inference. In *International Conference on Machine Learning*, pp. 812–821. PMLR, 2019.
- Cheon, J. H., Kim, A., Kim, M., and Song, Y. Homomorphic encryption for arithmetic of approximate numbers. In Takagi, T. and Peyrin, T. (eds.), *Advances in Cryptology – ASIACRYPT 2017*, pp. 409–437. Springer International Publishing, 2017. ISBN 978-3-319-70694-8.
- Cheon, J. H., Han, K., Kim, A., Kim, M., and Song, Y. Bootstrapping for approximate homomorphic encryption. In Nielsen, J. B. and Rijmen, V. (eds.), *Advances in Cryptology – EUROCRYPT 2018*, pp. 360–384, Cham, 2018a. Springer International Publishing. ISBN 978-3-319-78381-9.
- Cheon, J. H., Han, K., Kim, A., Kim, M., and Song, Y. A full RNS variant of approximate homomorphic encryption. *Selected areas in cryptography : annual international workshop, SAC proceedings. SAC*, 11349:347–368, 2018b.
- Cheon, J. H., Kim, D., and Kim, D. Efficient homomorphic comparison methods with optimal complexity. In *Advances in Cryptology—ASIACRYPT 2020: 26th International Conference on the Theory and Application of Cryptology and Information Security, Daejeon, South Korea, December 7–11, 2020, Proceedings, Part II 26*, pp. 221–256. Springer, 2020.
- Cheon, J. H., Cho, W., Kim, J., and Stehlé, D. Homomorphic multiple precision multiplication for ckks and reduced modulus consumption. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*, pp. 696–710, 2023.
- Chillotti, I., Gama, N., Georgieva, M., and Izabachène, M. Tfhe : fast fully homomorphic encryption over the torus. *Journal of Cryptology*, 33(1):34–91, 2020.
- Chou, E., Beal, J., Levy, D., Yeung, S., Haque, A., and Fei-Fei, L. Faster cryptonets: Leveraging sparsity for real-world encrypted inference. *arXiv preprint arXiv:1811.09953*, 2018.
- CryptoLab. HEaaN. <https://heaan.it>, ver. September 2022.
- Dathathri, R., Saarikivi, O., Chen, H., Laine, K., Lauter, K., Maleki, S., Musuvathi, M., and Mytkowicz, T. CHET: an optimizing compiler for fully-homomorphic neural network inferencing. In *Proceedings of the 40th ACM*

- SIGPLAN Conference on Programming Language Design and Implementation*, pp. 142–156, 2019.
- EPFL-LDS, T. I. S. Lattigo. <https://github.com/tuneinsight/lattigo>, 2022, v3.
- Fan, J. and Vercauteren, F. Somewhat practical fully homomorphic encryption. *Cryptology ePrint Archive*, 2012.
- Gentry, C. Fully homomorphic encryption using ideal lattices. In *Proceedings of the forty-first annual ACM symposium on Theory of computing*, pp. 169–178, 2009.
- Gentry, C., Halevi, S., and Smart, N. P. Homomorphic evaluation of the aes circuit. In *Lecture Notes in Computer Science*, pp. 850–867. Springer Berlin Heidelberg, 2012. doi: 10.1007/978-3-642-32009-5_49.
- Gilad-Bachrach, R., Dowlin, N., Laine, K., Lauter, K., Naehrig, M., and Wernsing, J. Cryptonets: Applying neural networks to encrypted data with high throughput and accuracy. In Balcan, M. F. and Weinberger, K. Q. (eds.), *Proceedings of The 33rd International Conference on Machine Learning*, volume 48 of *Proceedings of Machine Learning Research*, pp. 201–210, New York, New York, USA, 20–22 Jun 2016. PMLR. URL <https://proceedings.mlr.press/v48/gilad-bachrach16.html>.
- Han, K. and Ki, D. Better bootstrapping for approximate homomorphic encryption. In *Topics in Cryptology – CT-RSA 2020*, pp. 364–390. Springer International Publishing, 2020. doi: 10.1007/978-3-030-40186-3_16.
- Hesamifard, E., Takabi, H., and Ghasemi, M. Cryptodl: towards deep learning over encrypted data. In *Annual Computer Security Applications Conference (ACSAC 2016)*, Los Angeles, California, USA, volume 11, 2016.
- Huang, Z., Lu, W.-j., Hong, C., and Ding, J. Cheetah: Lean and fast secure two-party deep neural network inference. In *31st USENIX Security Symposium (USENIX Security 22)*, pp. 809–826, 2022.
- Jung, W., Kim, S., Ahn, J. H., Cheon, J. H., and Lee, Y. Over 100x faster bootstrapping in fully homomorphic encryption through memory-centric optimization with gpus. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, pp. 114–148, 2021.
- Juvekar, C., Vaikuntanathan, V., and Chandrakasan, A. GAZELLE: A low latency framework for secure neural network inference. In *Proceedings of the 27th USENIX Conference on Security Symposium, SEC’18*, pp. 1651–1668, USA, 2018. USENIX Association. ISBN 9781931971461.
- Kim, A., Papadimitriou, A., and Polyakov, Y. Approximate homomorphic encryption with reduced approximation error. In *Cryptographers’ Track at the RSA Conference*, pp. 120–144. Springer, 2022a.
- Kim, D. and Guyot, C. Optimized privacy-preserving cnn inference with fully homomorphic encryption. *IEEE Transactions on Information Forensics and Security*, 18: 2175–2187, 2023. doi: 10.1109/TIFS.2023.3263631.
- Kim, M., Jiang, X., Lauter, K., Ismayilzada, E., and Shams, S. Secure human action recognition by encrypted neural network inference. *Nature Communications*, 13(1), August 2022b. doi: 10.1038/s41467-022-32168-5.
- Kim, M., Lee, D., Seo, J., and Song, Y. Accelerating he operations from key decomposition technique. In *Annual International Cryptology Conference*, pp. 70–92. Springer, 2023.
- Krizhevsky, A. Learning multiple layers of features from tiny images, 2009. URL <https://www.cs.toronto.edu/~kriz/learning-features-2009-TR.pdf>.
- Lee, E., Lee, J.-W., Lee, J., Kim, Y.-S., Kim, Y., No, J.-S., and Choi, W. Low-complexity deep convolutional neural networks on fully homomorphic encryption using multiplexed parallel convolutions. In Chaudhuri, K., Jegelka, S., Song, L., Szepesvari, C., Niu, G., and Sabato, S. (eds.), *Proceedings of the 39th International Conference on Machine Learning*, volume 162 of *Proceedings of Machine Learning Research*, pp. 12403–12422. PMLR, 17–23 Jul 2022a.
- Lee, J., Lee, E., Lee, J.-W., Kim, Y., Kim, Y.-S., and No, J.-S. Precise approximation of convolutional neural networks for homomorphically encrypted data. *arXiv preprint arXiv:2105.10879*, 2021.
- Lee, J.-W., Kang, H., Lee, Y., Choi, W., Eom, J., Deryabin, M., Lee, E., Lee, J., Yoo, D., Kim, Y.-S., and No, J.-S. Privacy-preserving machine learning with fully homomorphic encryption for deep neural network. *IEEE Access*, 10:30039–30054, 2022b. doi: 10.1109/ACCESS.2022.3159694.
- Lee, J.-W., Lee, E., Kim, Y.-S., and No, J.-S. Hierarchical galois key management systems for privacy preserving aiaas with homomorphic encryption. *Cryptology ePrint Archive*, Paper 2022/532, 2022c.
- Lee, Y., Lee, J.-W., Kim, Y.-S., Kim, Y., No, J.-S., and Kang, H. High-precision bootstrapping for approximate homomorphic encryption by error variance minimization. In Dunkelman, O. and Dziembowski, S. (eds.), *Advances in Cryptology – EUROCRYPT 2022*, pp. 551–580, Cham,

2022d. Springer International Publishing. ISBN 978-3-031-06944-4.

Legiest, W., Turan, F., Van Beirendonck, M., D’Anvers, J.-P., and Verbauwhede, I. Neural network quantisation for faster homomorphic encryption. In *2023 IEEE 29th International Symposium on On-Line Testing and Robust System Design (IOLTS)*, pp. 1–3. IEEE, 2023.

Lou, Q. and Jiang, L. She: A fast and accurate deep neural network for encrypted data. *Advances in neural information processing systems*, 32, 2019.

Lou, Q., Lu, W.-j., Hong, C., and Jiang, L. Falcon: Fast spectral inference on encrypted data. *Advances in Neural Information Processing Systems*, 33:2364–2374, 2020.

Micali, S., Goldreich, O., and Wigderson, A. How to play any mental game. In *Proceedings of the Nineteenth ACM Symp. on Theory of Computing, STOC*, pp. 218–229. ACM New York, NY, USA, 1987.

Mishra, P., Lehmkuhl, R., Srinivasan, A., Zheng, W., and Popa, R. A. Delphi: A cryptographic inference system for neural networks. In *Proceedings of the 2020 Workshop on Privacy-Preserving Machine Learning in Practice*, pp. 27–30, 2020.

Ng, L. K. and Chow, S. S. Gforce:gpu-friendly oblivious and rapid neural network inference. In *30th USENIX Security Symposium (USENIX Security 21)*, pp. 2147–2164, 2021.

Ng, L. K. and Chow, S. S. Sok: cryptographic neural-network computation. In *2023 IEEE Symposium on Security and Privacy (SP)*, pp. 497–514. IEEE, 2023.

Rathee, D., Rathee, M., Goli, R. K. K., Gupta, D., Sharma, R., Chandran, N., and Rastogi, A. Sirnn: A math library for secure rnn inference. In *2021 IEEE Symposium on Security and Privacy (SP)*, pp. 1003–1020. IEEE, 2021.

Sanyal, A., Kusner, M., Gascon, A., and Kanade, V. Tapas: Tricks to accelerate (encrypted) prediction as a service. In *International conference on machine learning*, pp. 4490–4499. PMLR, 2018.

SEAL. Microsoft seal (release 4.1.1). <https://github.com/Microsoft/SEAL>, January 2023.

Snu-ccl. FHE-MP-CNN. <https://github.com/snu-ccl/FHE-MP-CNN>, 2022.

Yao, A. C.-C. How to generate and exchange secrets. In *27th annual symposium on foundations of computer science (Sfcs 1986)*, pp. 162–167. IEEE, 1986.